



Universidad Nacional de Costa Rica

Facultad de Ciencias Exactas y Naturales

Escuela de Informatica

Proyecto II

Documento Tecnico

Simulacion de Mazmorra

Curso: EIF204 - Programacion II

Profesor: Luis Elias Mora Montero

Lenguaje: C++20

Entorno de desarrollo: CLion / CMake

Integrantes:

Chris Martinez Sanchez – 119550022

Jimena Valencia Montero – 402710618

Índice

1. Introduccion	3
2. Descripcion General del Sistema	3
3. Tematica de la Aventura	4
4. Arquitectura del Sistema	4
4.1. Modulo Engine	4
4.2. Modulo World	4
4.3. Modulo Entities	4
4.4. Modulo Items	5
4.5. Modulo Events	5
4.6. Modulo Objectives	5
4.7. Modulo Utilities	5
4.8. Modulo Exceptions	5
5. Diagrama UML	6
6. Formato de los Archivos de Entrada	6
6.1. Archivo rooms.txt	6
6.2. Archivo connections.txt	7
6.3. Archivo items.txt	7
6.4. Archivo enemies.txt	7
7. Principales Clases y Relaciones	7
7.1. AdventureEngine	7
7.2. SimulationEngine	8
7.3. World y Room	8
7.4. Character, Player y Enemy	8
7.5. Item, Potion, Key, Treasure e Inventory	8
7.6. GameEvent, TrapEvent y RewardEvent	8
7.7. Objective	8
7.8. FileManager, Logger y ReportGenerator	8
8. Decisiones Relevantes de Diseno	8
9. Estrategia de Manejo de Memoria y Recursos	9

10.Manejo de Archivos y Errores	9
10.1. Manejo de archivos	9
10.2. Manejo de errores	10
11.Tecnicas del Curso Utilizadas y Justificacion	10
11.1. Herencia	10
11.2. Polimorfismo	10
11.3. Encapsulamiento	10
11.4. Composicion	10
11.5. Manejo de archivos	11
11.6. Excepciones	11
11.7. Smart pointers	11
12.Evidencia de Ejecucion	11
12.1. Captura de ejecucion en consola	11
12.2. Salida de consola	11
12.3. Captura de carpeta data	13
12.4. Captura de carpeta output	14
12.5. Bitacora generada	15
12.6. Reporte final generado	16
13.Resultados Obtenidos	16
14.Limitaciones del Sistema	17
15.Posibles Mejoras	17
16.Conclusiones	17

1 Introduccion

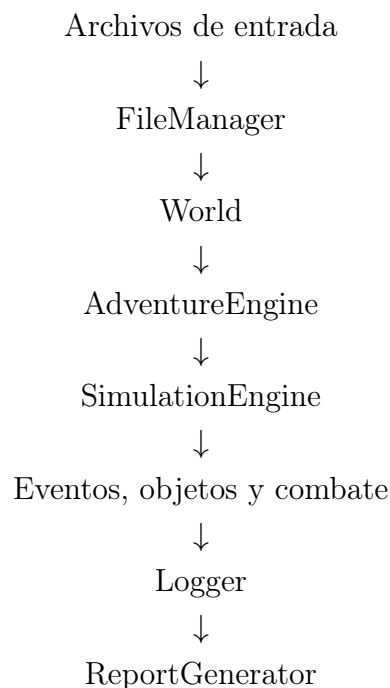
El presente documento tecnico describe el diseno, construccion y funcionamiento del proyecto desarrollado para el curso EIF204 Programacion II. La solucion consiste en una simulacion de aventura dentro de una mazmorra, implementada en C++ utilizando principios de Programacion Orientada a Objetos.

El proposito del proyecto es integrar en una misma aplicacion conceptos fundamentales del curso, tales como herencia, polimorfismo, encapsulamiento, composicion, manejo de archivos, manejo de excepciones, uso de memoria dinamica y separacion modular del codigo.

2 Descripcion General del Sistema

El sistema desarrollado representa una aventura de exploracion dentro de una mazmorra. El jugador principal recorre distintas habitaciones, recolecta objetos, activa eventos especiales, enfrenta enemigos y completa un objetivo principal.

El flujo general de ejecucion es el siguiente:



La clase **AdventureEngine** funciona como controlador principal. Esta coordina la carga de datos mediante **FileManager**, la ejecucion de la simulacion mediante **SimulationEngine** y la generacion del reporte final mediante **ReportGenerator**.

3 Tematica de la Aventura

La aventura se desarrolla dentro de una antigua mazmorra compuesta por habitaciones conectadas entre si. El jugador principal, Chris, inicia la exploracion acompañado por Jimena.

Durante el recorrido se visitan distintas areas de la mazmorra, tales como la entrada, un pasillo oscuro, una sala del tesoro y la guarida del dragon. En estas habitaciones el jugador puede encontrar objetos como pociones, llaves y tesoros.

La aventura culmina con un combate contra Drogon, un dragon que representa el enemigo principal. El objetivo es sobrevivir al recorrido, recolectar objetos importantes y derrotar al enemigo final.

4 Arquitectura del Sistema

El proyecto fue organizado en modulos para separar responsabilidades y facilitar el mantenimiento del codigo.

4.1 Modulo Engine

Contiene las clases que coordinan la ejecucion del sistema.

- **AdventureEngine**: controlador principal del sistema.
- **SimulationEngine**: ejecuta la logica de la simulacion.

4.2 Modulo World

Contiene las clases relacionadas con el mundo de la aventura.

- **World**: administra habitaciones, jugador y eventos.
- **Room**: representa una habitacion de la mazmorra.

4.3 Modulo Entities

Contiene las clases relacionadas con personajes.

- **Character**: clase base de personajes.
- **Player**: representa al jugador.
- **Enemy**: representa enemigos.

4.4 Modulo Items

Contiene las clases relacionadas con objetos.

- **Item**: clase base abstracta para objetos.
- **Potion**: objeto que recupera vida.
- **Key**: objeto que representa una llave.
- **Treasure**: objeto que otorga puntos.
- **Inventory**: administra objetos recolectados.

4.5 Modulo Events

Contiene las clases relacionadas con eventos especiales.

- **GameEvent**: clase base abstracta de eventos.
- **TrapEvent**: evento que causa dano.
- **RewardEvent**: evento que otorga recompensas.

4.6 Modulo Objectives

Contiene la clase **Objective**, encargada de validar si el objetivo principal de la aventura fue completado.

4.7 Modulo Utilities

Contiene clases auxiliares.

- **FileManager**: creacion y lectura de archivos.
- **Logger**: registro de acontecimientos.
- **ReportGenerator**: generacion del reporte final.

4.8 Modulo Exceptions

Contiene excepciones personalizadas.

- **GameException**
- **FileException**
- **InvalidDataException**
- **SimulationException**

5 Diagrama UML

En esta seccion se debe insertar el diagrama UML final del proyecto. El diagrama debe mostrar las clases principales, relaciones de herencia, composicion y dependencias entre modulos.

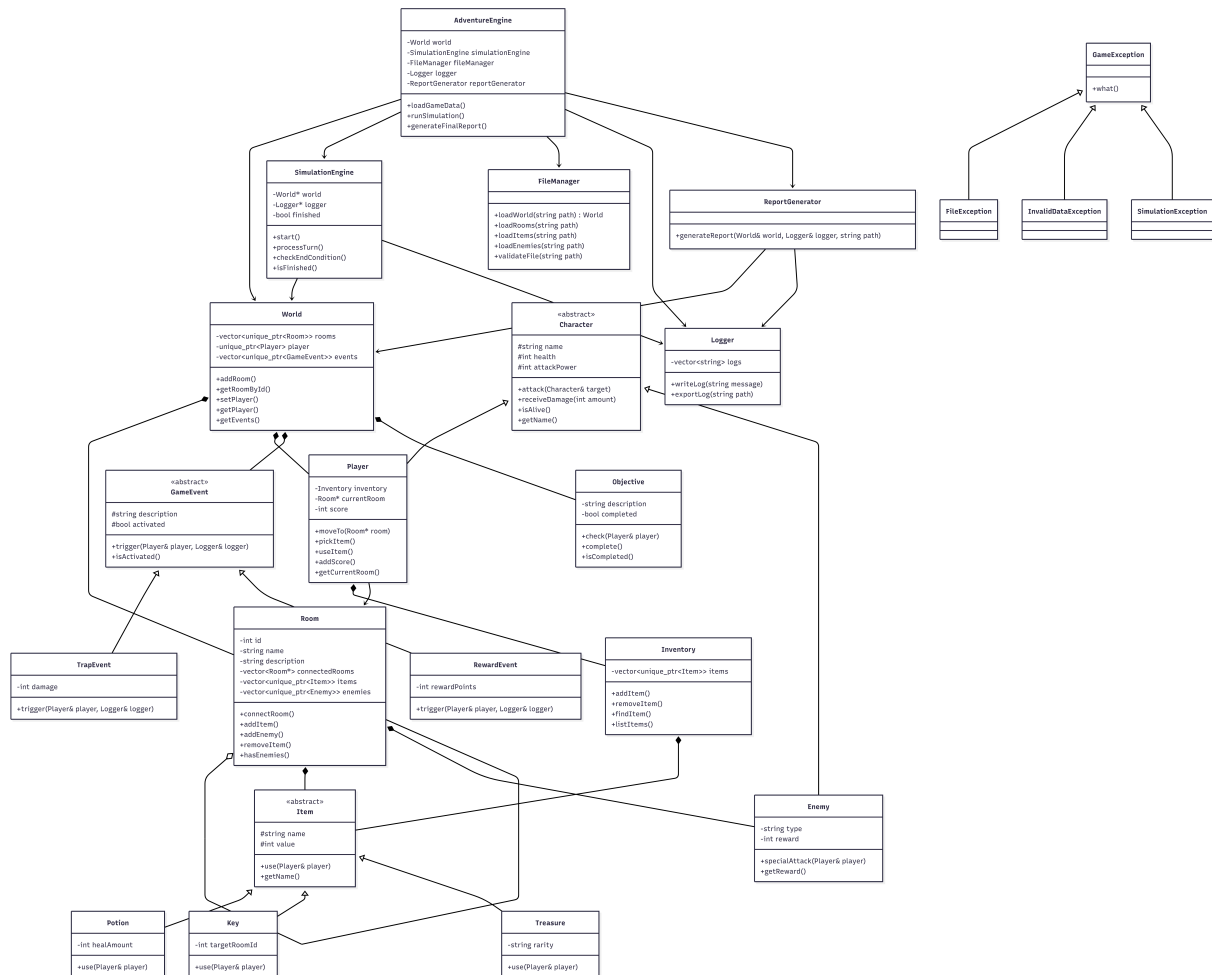


Figura 1: Diagrama UML del sistema

6 Formato de los Archivos de Entrada

El sistema utiliza archivos de texto separados por el caracter |. Estos archivos son creados automaticamente si no existen.

6.1 Archivo rooms.txt

Este archivo define las habitaciones del mundo.

```

1|Entrance|Inicio de la mazmorra
2|Hallway|Pasillo oscuro
3|Treasure Room|Sala del tesoro
  
```

```
4|Dragon Lair|Guarida del dragon
```

Formato:

```
id|nombre|descripcion
```

6.2 Archivo connections.txt

Este archivo define las conexiones entre habitaciones.

```
1|2  
2|3  
3|4
```

Formato:

```
idHabitacionOrigen|idHabitacionDestino
```

6.3 Archivo items.txt

Este archivo define los objetos disponibles en las habitaciones.

```
1|Potion|Small Potion|25|30  
3|Treasure|Ancient Crown|150|Legendary  
3|Key|Golden Key|50|4
```

Formato:

```
idHabitacion|tipoObjeto|nombre|valor|atributoEspecial
```

6.4 Archivo enemies.txt

Este archivo define los enemigos presentes en el mundo.

```
4|Drogon|Dragon|45|25|150
```

Formato:

```
idHabitacion|nombre|tipo|vida|ataque|recompensa
```

7 Principales Clases y Relaciones

7.1 AdventureEngine

La clase `AdventureEngine` es el controlador principal del sistema. Su funcion es coordinar la carga de datos, ejecucion de la simulacion y generacion de reportes. Esta clase se relaciona con `World`, `FileManager`, `Logger`, `ReportGenerator` y `SimulationEngine`.

7.2 SimulationEngine

La clase `SimulationEngine` ejecuta la logica de la aventura. Administra el recorrido, los eventos, el combate y el objetivo principal.

7.3 World y Room

`World` representa el mundo completo. Contiene habitaciones y el jugador principal. `Room` representa una habitacion individual y almacena conexiones, objetos y enemigos.

7.4 Character, Player y Enemy

`Character` es la clase base para personajes. `Player` y `Enemy` heredan de esta clase, reutilizando atributos y metodos comunes como nombre, vida, poder de ataque y comportamiento de ataque.

7.5 Item, Potion, Key, Treasure e Inventory

`Item` es una clase abstracta para objetos. Sus clases derivadas implementan comportamientos concretos mediante el metodo `use`. `Inventory` administra los objetos recolectados durante la simulacion.

7.6 GameEvent, TrapEvent y RewardEvent

`GameEvent` representa un evento general. `TrapEvent` y `RewardEvent` implementan eventos concretos que modifican el estado del jugador.

7.7 Objective

La clase `Objective` se encarga de comprobar si se completo el objetivo principal de la aventura.

7.8 FileManager, Logger y ReportGenerator

`FileManager` administra archivos de entrada, `Logger` registra eventos de ejecucion y `ReportGenerator` genera el reporte final.

8 Decisiones Relevantes de Diseno

Una decision importante fue dividir el sistema en modulos. Esto permite separar responsabilidades y reducir el acoplamiento entre clases.

Tambien se decidio utilizar clases abstractas para representar conceptos generales como personajes, objetos y eventos. Esta decision permite aplicar polimorfismo y facilita extender el sistema en el futuro.

Otra decision relevante fue construir el mundo desde archivos de texto. Esto permite modificar habitaciones, conexiones, objetos y enemigos sin modificar el codigo fuente.

Ademas, se implemento generacion automatica de archivos y carpetas, lo que facilita la ejecucion del sistema en diferentes computadoras.

Finalmente, se utilizo **AdventureEngine** como controlador principal para que el metodo **main** no concentre toda la logica del sistema.

9 Estrategia de Manejo de Memoria y Recursos

El sistema utiliza `std::unique_ptr` para administrar memoria dinamica. Esta estrategia permite que la memoria sea liberada automaticamente cuando los objetos salen de alcance.

Se utilizo `unique_ptr` en los siguientes casos:

- Habitaciones almacenadas en `World`.
- Objetos almacenados en `Room`.
- Enemigos almacenados en `Room`.
- Jugador principal almacenado en `World`.
- Objetos almacenados en `Inventory`.

Esta decision evita fugas de memoria y elimina la necesidad de utilizar `delete` manualmente.

10 Manejo de Archivos y Errores

10.1 Manejo de archivos

El sistema utiliza `FileManager` para crear, validar y cargar los archivos de entrada. Si los archivos no existen, el sistema los genera automaticamente.

Tambien se utiliza `std::filesystem` para trabajar con rutas relativas. Esto permite que el sistema funcione en diferentes computadoras sin modificar rutas absolutas.

Las carpetas utilizadas son:

- `data`: contiene archivos de entrada.
- `output`: contiene archivos generados.

10.2 Manejo de errores

Se implementaron excepciones personalizadas para representar errores del dominio del proyecto:

- `GameException`: excepcion base del sistema.
- `FileNotFoundException`: errores relacionados con archivos.
- `InvalidDataException`: errores por datos invalidos.
- `SimulationException`: errores durante la simulacion.

Ademas, se validan casos como nombres vacios, objetos nulos y datos inconsistentes.

11 Tecnicas del Curso Utilizadas y Justificacion

11.1 Herencia

Se aplico herencia para reutilizar atributos y comportamientos comunes.

- `Character` → `Player`, `Enemy`.
- `Item` → `Potion`, `Key`, `Treasure`.
- `GameEvent` → `TrapEvent`, `RewardEvent`.

11.2 Polimorfismo

El polimorfismo se utiliza al trabajar con objetos derivados de `Item` y `GameEvent` mediante interfaces comunes.

11.3 Encapsulamiento

Los atributos de las clases se mantienen privados o protegidos, accediendo a ellos mediante metodos publicos.

11.4 Composicion

Se aplico composicion en relaciones como:

- `World` contiene habitaciones.
- `Room` contiene objetos y enemigos.
- `SimulationEngine` contiene eventos y objetivo.

11.5 Manejo de archivos

Se utilizo lectura y escritura de archivos para cargar el mundo y generar resultados.

11.6 Excepciones

Las excepciones permiten controlar errores de forma segura sin detener abruptamente la aplicacion.

11.7 Smart pointers

Se utilizo `std::unique_ptr` para administrar recursos dinamicos automaticamente.

12 Evidencia de Ejecucion

12.1 Captura de ejecucion en consola

```

===== SIMULACION DE AVENTURA =====

Jugador: Chris | Vida: 100 | Ataque: 15
Companera: Jimena | Vida: 100 | Ataque: 15

===== RECORRIDO POR HABITACIONES =====
Chris entra a Entrance
Chris encuentra Small Potion
Chris se mueve a Hallway
Chris se mueve a Treasure Room
Chris encuentra Ancient Crown
Chris encuentra Golden Key

===== INVENTARIO =====
- Small Potion | Valor: 25
- Ancient Crown | Valor: 150
- Golden Key | Valor: 50

===== USO DE OBJETOS Y EVENTOS =====
Chris activa una trampa, vida actual: 60
Chris usa Small Potion, vida actual: 90
Chris usa Ancient Crown, puntaje actual: 150
Chris usa Golden Key

Chris se mueve a Dragon Lair

===== COMBATE =====
Chris encuentra a Drogon
Chris ataca a Drogon | Vida de Drogon: 30
Jimena ataca a Drogon | Vida de Drogon: 15
Drogon usa ataque especial | Vida de Chris: 60
Chris ataca a Drogon | Vida de Drogon: 0

Drogon fue derrotado
Puntaje final de Chris: 300

===== PRUEBAS DE ERRORES =====
[OK] Error detectado: El nombre del personaje no puede estar vacio
[OK] Error detectado: El tipo de enemigo no puede estar vacio
[OK] Error detectado: No se puede agregar un objeto nulo al inventario
[OK] GameException detectada: Prueba de excepcion de simulacion

===== ARCHIVOS GENERADOS =====
Datos: C:\Users\chris\Desktop\Proyecto2\data
Bitacora: C:\Users\chris\Desktop\Proyecto2\output\log.txt
Reporte final: C:\Users\chris\Desktop\Proyecto2\output\final_report.txt

===== PRUEBAS FINALIZADAS =====

Process finished with exit code 0

```

Figura 2: Ejecucion del sistema en consola

12.2 Salida de consola

```

===== SIMULACION DE AVENTURA =====

Jugador: Chris | Vida: 100 | Ataque: 15
Companera: Jimena | Vida: 100 | Ataque: 15

```

```
===== RECORRIDO POR HABITACIONES =====
Chris entra a Entrance
Chris encuentra Small Potion
Chris se mueve a Hallway
Chris se mueve a Treasure Room
Chris encuentra Ancient Crown
Chris encuentra Golden Key

===== INVENTARIO =====
- Small Potion | Valor: 25
- Ancient Crown | Valor: 150
- Golden Key | Valor: 50

===== USO DE OBJETOS Y EVENTOS =====
Chris activa una trampa, vida actual: 60
Chris usa Small Potion, vida actual: 90
Chris usa Ancient Crown, puntaje actual: 150
Chris usa Golden Key

Chris se mueve a Dragon Lair

===== COMBATE =====
Chris encuentra a Drogon
Chris ataca a Drogon | Vida de Drogon: 30
Jimena ataca a Drogon | Vida de Drogon: 15
Drogon usa ataque especial | Vida de Chris: 60
Chris ataca a Drogon | Vida de Drogon: 0

Drogon fue derrotado
Puntaje final de Chris: 300

===== PRUEBAS DE ERRORES =====
[OK] Error detectado: El nombre del personaje no puede estar vacio
[OK] Error detectado: El tipo de enemigo no puede estar vacio
[OK] Error detectado: No se puede agregar un objeto nulo al inventario
[OK] GameException detectada: Prueba de excepcion de simulacion

===== ARCHIVOS GENERADOS =====
Datos: C:\Users\chris\Desktop\Proyecto2\data
Bitacora: C:\Users\chris\Desktop\Proyecto2\output\log.txt
Reporte final: C:\Users\chris\Desktop\Proyecto2\output\final_report.txt

===== PRUEBAS FINALIZADAS =====
```

12.3 Captura de carpeta data

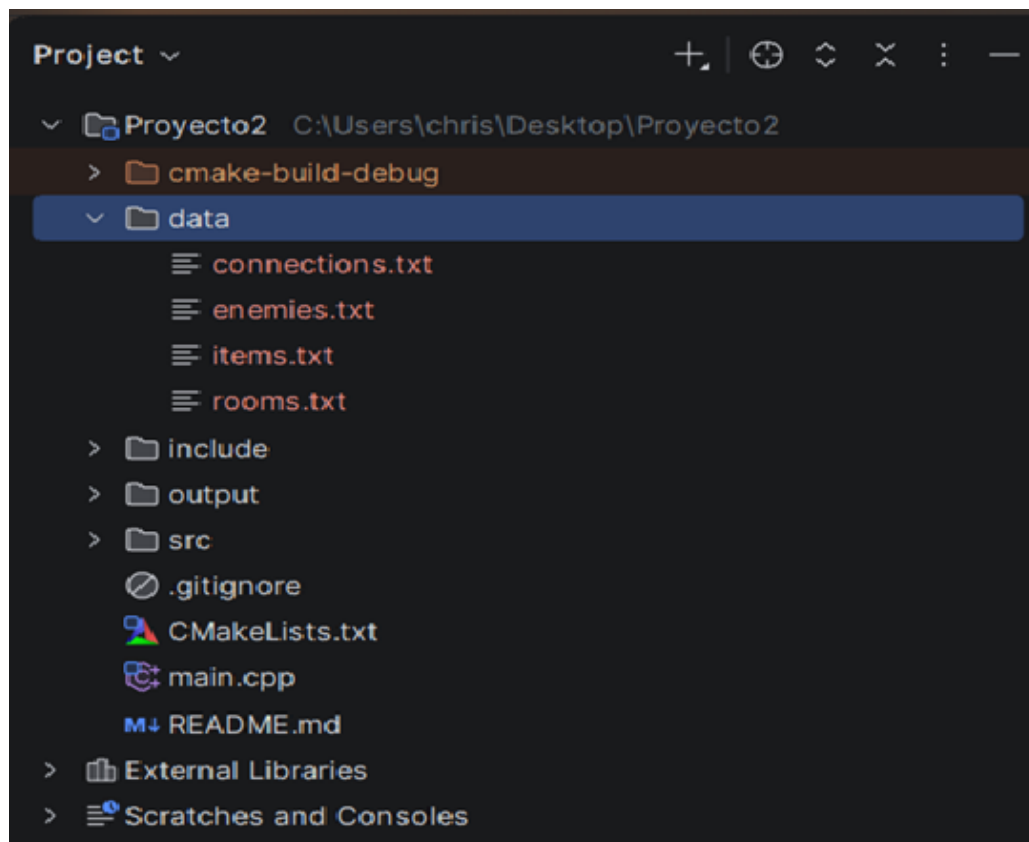


Figura 3: Archivos de entrada generados en la carpeta data

12.4 Captura de carpeta output

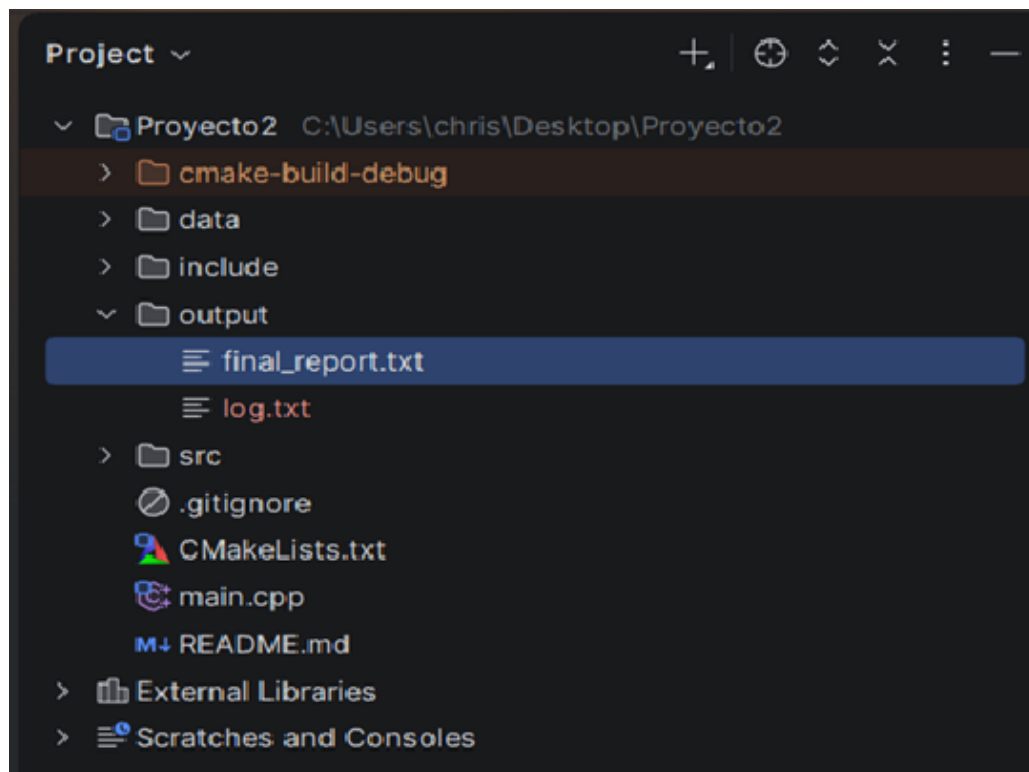
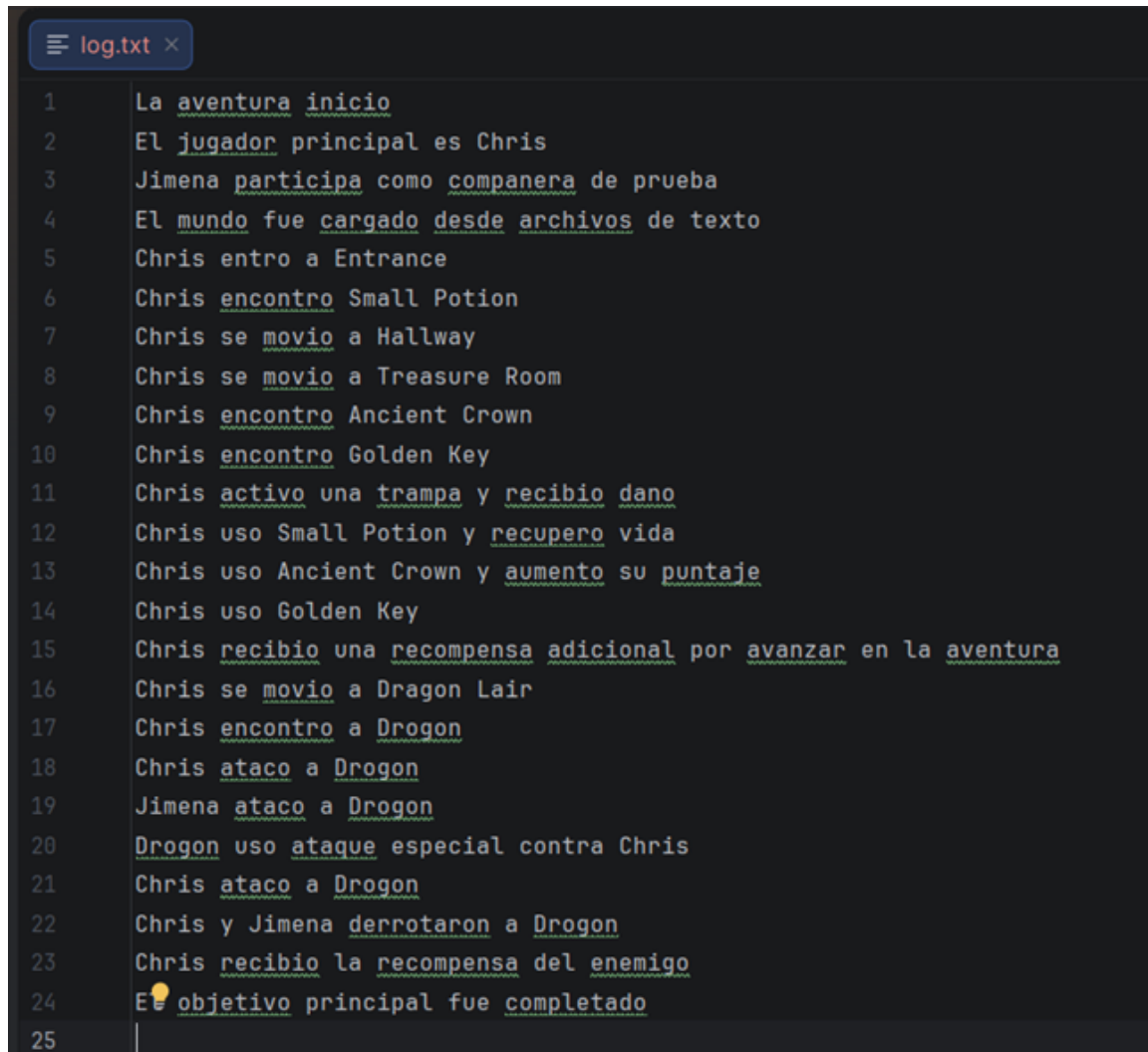


Figura 4: Archivos de salida generados en la carpeta output

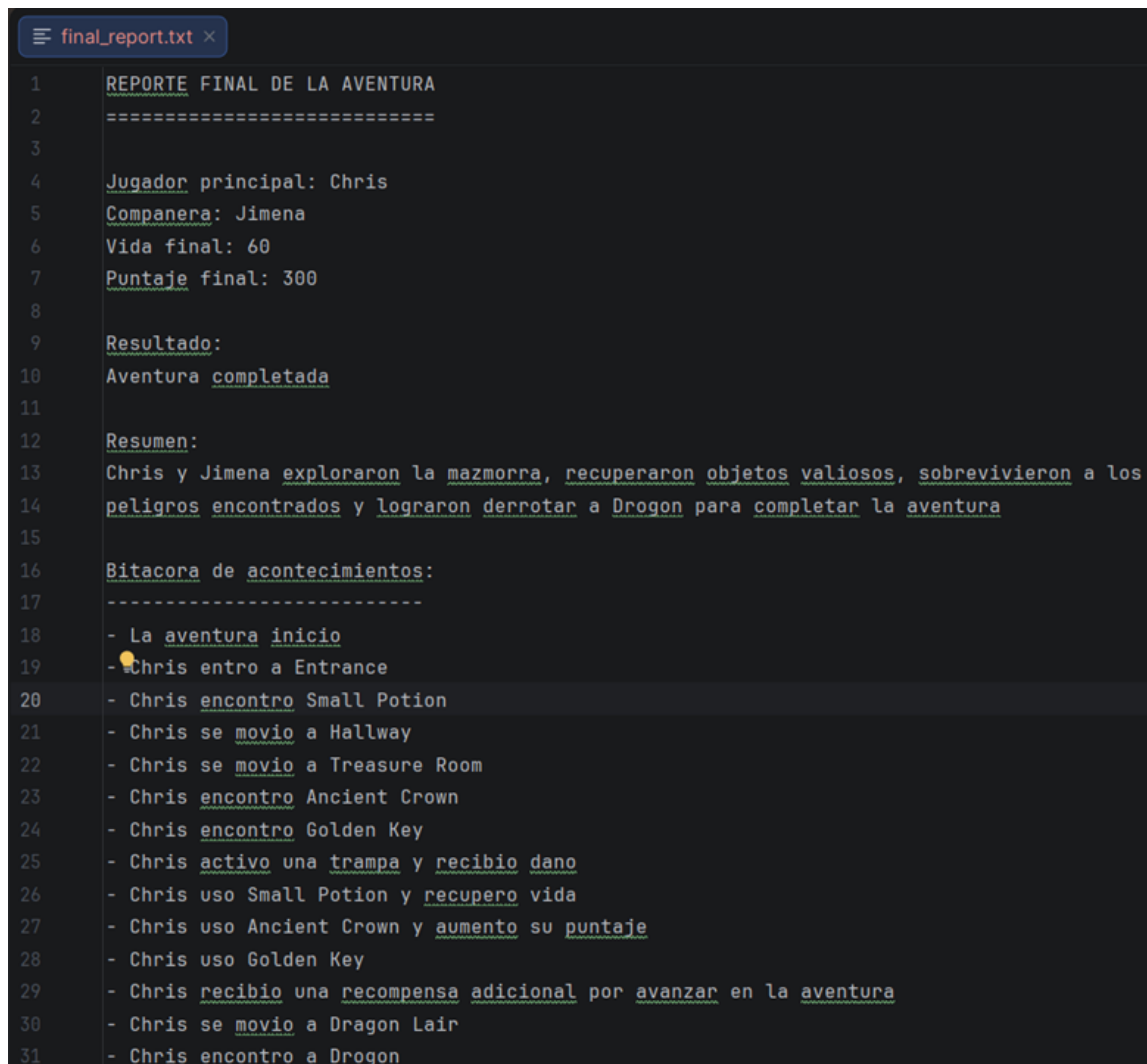
12.5 Bitacora generada



```
log.txt x
1  La aventura inicio
2  El jugador principal es Chris
3  Jimena participa como companera de prueba
4  El mundo fue cargado desde archivos de texto
5  Chris entro a Entrance
6  Chris encontro Small Potion
7  Chris se movio a Hallway
8  Chris se movio a Treasure Room
9  Chris encontro Ancient Crown
10 Chris encontro Golden Key
11 Chris activo una trampa y recibio dano
12 Chris uso Small Potion y recupero vida
13 Chris uso Ancient Crown y aumento su puntaje
14 Chris uso Golden Key
15 Chris recibio una recompensa adicional por avanzar en la aventura
16 Chris se movio a Dragon Lair
17 Chris encontro a Drogon
18 Chris ataco a Drogon
19 Jimena ataco a Drogon
20 Drogon uso ataque especial contra Chris
21 Chris ataco a Drogon
22 Chris y Jimena derrotaron a Drogon
23 Chris recibio la recompensa del enemigo
24 El objetivo principal fue completado
25
```

Figura 5: Contenido del archivo log.txt

12.6 Reporte final generado



```
1  REPORTE FINAL DE LA AVENTURA
2  =====
3
4  Jugador principal: Chris
5  Companera: Jimena
6  Vida final: 60
7  Puntaje final: 300
8
9  Resultado:
10 Aventura completada
11
12 Resumen:
13 Chris y Jimena exploraron la mazmorra, recuperaron objetos valiosos, sobrevivieron a los
14 peligros encontrados y lograron derrotar a Drogon para completar la aventura
15
16 Bitacora de acontecimientos:
17 -----
18 - La aventura inicio
19 - Chris entro a Entrance
20 - Chris encontro Small Potion
21 - Chris se movio a Hallway
22 - Chris se movio a Treasure Room
23 - Chris encontro Ancient Crown
24 - Chris encontro Golden Key
25 - Chris activo una trampa y recibio dano
26 - Chris uso Small Potion y recupero vida
27 - Chris uso Ancient Crown y aumento su puntaje
28 - Chris uso Golden Key
29 - Chris recibio una recompensa adicional por avanzar en la aventura
30 - Chris se movio a Dragon Lair
31 - Chris encontro a Drogon
```

Figura 6: Contenido del archivo final_report.txt

13 Resultados Obtenidos

Durante las pruebas se verifico que el sistema:

- Carga informacion desde archivos de texto.
- Construye el mundo de juego correctamente.
- Permite recorrer habitaciones.
- Administra objetos mediante inventario.
- Ejecuta eventos especiales.
- Realiza combate contra enemigos.

- Valida errores mediante excepciones.
- Genera bitacora de acontecimientos.
- Genera reporte final.

14 Limitaciones del Sistema

El sistema cumple con los objetivos principales, pero presenta algunas limitaciones:

- El recorrido de la aventura es predefinido.
- El mapa contiene pocas habitaciones.
- Existe un unico objetivo principal.
- Los enemigos tienen comportamiento basico.
- No existe interfaz grafica.

15 Posibles Mejoras

Entre las posibles mejoras se encuentran:

- Permitir interaccion directa del usuario.
- Agregar mapas mas grandes.
- Implementar enemigos con comportamiento mas avanzado.
- Agregar multiples objetivos.
- Implementar guardado y carga de partidas.
- Agregar mas tipos de eventos.
- Crear una interfaz grafica.

16 Conclusiones

El proyecto permitio aplicar de forma integrada los principales conceptos estudiados en el curso de Programacion II.

La solucion desarrollada demuestra el uso de programacion orientada a objetos, herencia, polimorfismo, composicion, manejo de archivos, excepciones y administracion automatica de memoria mediante `std::unique_ptr`.

La estructura modular facilita el mantenimiento del sistema y permite que el proyecto pueda ampliarse en el futuro con nuevas habitaciones, enemigos, objetos, eventos y objetivos.